Hey chat! Today you will be my study buddy. I have an uncomming lecture today on "In-context learning, prompting and PEFT: Soft-prompting and LoRA". give me some prelecture notes that i can read before lecture so I go into it not completely lost.

Perfect — let's get you confidently warmed up before your lecture. Before I tailor it:

👉 How deep should I go? For example:

- ☐ **Quick overview (10–15 min read)** — plain-language summary with key ideas and intuition.
- ☐ **Medium (30–45 min read)** — include some formulas, architecture sketches, and pros/cons.
- ☐ **Deep dive (1–2 hr read)** — technical treatment of how LoRA and soft prompts work internally, parameter-efficient math, and implementation notes.

Also, are you familiar with **basic transformer architecture and fine-tuning** concepts already (e.g. attention, pretraining, fine-tuning)?

Once I know that, I'll build you a crisp, structured prelecture guide (with small comprehension checks to help retention).

Great question. Let's go for a quick overview, just so I can get a little familiar with ideas and not be completely lost. Also thanks for asking. We already covered some attention, pretraining but not much of fine-tuning. Hope that helps!

Perfect — thanks, that's exactly what I needed.
Here's our **plan for today's prelecture warm-up**:

✅ Step 1: Get the big picture — what "in-context learning," "prompting," and "PEFT" actually mean.

✅ Step 2: Meet the two PEFT stars — **Soft Prompting** and **LoRA** — and what problem they solve.

✅ Step 3: Do a short "check-your-understanding" quiz so you walk into class already connecting dots.

Let's start small.

🧠 Step 1. The Big Picture: What's going on here?

Large language models (LLMs) can adapt to new tasks in **two main ways**:

1. **In-context learning (ICL):**
   - You don't change the model weights.
   - You simply *show* the model examples or instructions *in the input context*.
   - Example:

     bash Copy code

     ```
     Translate English to French:
     cat → chat
     dog → chien
     house →
     ```

     The model continues with "maison."
   - Think of it as "temporary learning" during inference — like showing examples on a whiteboard.
2. **Fine-tuning:**
   - You *update* model parameters using new training data to make it permanently better for a task.
   - Full fine-tuning of big models is expensive (billions of parameters!).

That's where **PEFT (Parameter-Efficient Fine-Tuning)** comes in — it tries to get the *benefits of fine-tuning* without updating every single parameter.

👉 Your turn:

Why might updating *only a small fraction* of parameters (instead of the whole model) be a good idea?

(Think cost, storage, or overfitting.)

> Probably since you don't have to update so many parameters which would take time, energy, and therefore money to do. By keeping the rest of the parameters untouched improving your model becomes cheaper.

Exactly right — nice reasoning.

✅ **You caught the key points:**

- **Compute/storage efficiency:** updating only a few parameters = less GPU memory and faster training.
- **Reusability:** you can keep one base model and just plug in small "adapter" pieces for different tasks.
- **Stability:** smaller updates reduce the risk of "catastrophic forgetting" (losing the original skills).

⚙️ Step 2. Enter PEFT Methods: "Soft Prompting" and "LoRA"

Let's sketch what they do **at a high level**:

1. **Soft Prompting (a.k.a. prompt tuning):**
   - Instead of adding *words* to your text prompt, you add *learnable embeddings* (tiny trainable vectors) to the model's input.
   - These embeddings act like an "invisible prefix" that guides the model's behavior.
   - The base model's weights never change — only these few vectors get trained.
   - Analogy: imagine slipping a small instruction note *inside* the model's head before it reads your text. 💬
2. **LoRA (Low-Rank Adaptation):**
   - Instead of touching all weights, you insert **tiny trainable matrices** inside existing layers.
   - These matrices learn task-specific tweaks in a low-dimensional way (low-rank = fewer parameters).
   - The base weights stay frozen; only the LoRA layers learn.
   - Analogy: you attach "smart tuning knobs" to the model's existing layers.

Both methods are “add-on adapters” that make fine-tuning cheap, modular, and reversible.

🧩 Mini check:
If a model uses **soft prompting**, where are the new trainable parameters located — in the main layers or somewhere else?
(Think about *what* gets trained here.)

In soft prompting, the new trainable parameters are located in the input

Perfect — that’s exactly it. 🎯

✅ **Soft prompting** only trains a few *embedding vectors* added to the model’s **input space**, not the internal transformer layers.
You can think of it as *teaching the model a new “context code”* rather than rewiring its internals.

Let’s lock this in with one more key comparison before your lecture.

Method	What’s trained	Where changes happen	Typical size	Pros
Soft Prompting	A few embedding vectors	Input layer	~0.1–1% of total params	Simple, quick, no architecture change
LoRA	Tiny matrices (low-rank adapters)	Inside attention & feedforward layers	~0.5–2%	Stronger adaptation, modular “plug-in” weights

---

🧠 Quick reflection before we wrap up:

Why might **LoRA** usually adapt better than **soft prompting**, especially on more complex downstream tasks?

(Think about *where* in the network each one can influence computation.)

Perhaps since you can add a matrix that contains more information (can be a low to medium rank if want to) and also since it is inside the attention layer

Exactly — excellent reasoning again.

✅ **LoRA** can influence the **internal transformations** (like attention and MLP layers) where deep representations form.

That gives it *more expressive power* than soft prompts, which only nudge the model through the input embeddings.

And as you said, those low-rank matrices can still encode rich information while staying lightweight — a nice middle ground between soft prompting and full fine-tuning.

⚡ Step 3. Recap & Quick Quiz

Recap in one breath:

- *In-context learning* = model "learns" from examples in the prompt (no weight updates).
- *Prompting* = guiding behavior through inputs.
- *PEFT* = fine-tuning efficiently by training only small parts.
- *Soft prompting* = learnable input embeddings.
- *LoRA* = low-rank adapters inside transformer layers.

Mini Quiz (2 mins):

1. Why is in-context learning called "learning" even though no parameters are updated?
2. In LoRA, what does "low-rank" mean in simple terms?

3. Which PEFT method would you pick if you want *fastest adaptation* with *minimal storage* (and you don't need perfect performance)?

Once you try those, I'll confirm and wrap you up with a short "lecture survival checklist" (what to focus on when the professor starts going technical).

1. the model learns from specific examples in the prompt. helps emphasize patterns and most importantly instruction learning.
2. the rank of the matrix that you input to learn. it is low rank so it is cheap and only helps adjust a small subset of parameters in the model.
3. soft-prompting

Perfectly done — you've got the intuition spot on across all three:

✅ **(1)** Right — *in-context learning* is called "learning" because the model *acts* like it's learning patterns from examples in the prompt, even though no weights change. It's temporary, on-the-fly reasoning.

✅ **(2)** Exactly — *low-rank* means the adapter matrix has limited independent directions (low dimensionality). That's what keeps LoRA efficient but expressive.

✅ **(3)** Correct again — *soft prompting* is the fastest, smallest, and most storage-friendly, though not as powerful as LoRA for complex tasks.

🧭 Lecture Survival Checklist

Here's what to listen for when your professor dives in:

- **How in-context learning** differs from *fine-tuning* at the parameter level.
- **Prompt types:** hard (text) vs. soft (learned embeddings).
- **PEFT methods:** LoRA, prefix-tuning, adapters — how each trades off *efficiency vs. flexibility.*

- **LoRA math idea:** $W' = W + AB^T$ (where A and B are low-rank matrices you train).
- **Evaluation theme:** how small updates can match full fine-tuning performance on many benchmarks.

Would you like me to make a **one-page printable summary sheet** (clean layout, bullet notes, 2-min prelecture glance)?
I can format it for fast recall before class.

sure let's do it

Perfect — before I generate it: would you like it as

- ☐ **Plain text / markdown** (for quick review on screen), or
- ☐ **PDF summary sheet** (for printing or annotating)?

And one more quick choice: should it emphasize

- ☐ **Concepts & intuition**, or
- ☐ **Key formulas & structure** (a bit more technical, good for future revision)?

pdf, specified for prelecture notes. just to reiterate

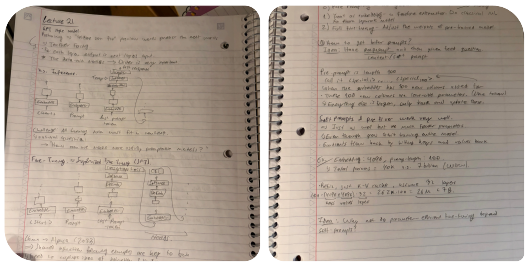
Here's your printable prelecture summary sheet:
Download Prelecture Notes – In-Context Learning, Prompting & PEFT (Soft Prompting + LoRA)

It's optimized for a quick read before class — includes intuition, comparison table, and key focus checklist. Would you like me to make a matching **post-lecture reflection sheet** (to fill in right after class to reinforce recall)?

that's it for now i'm going to lecture thanks!

Perfect — you're heading in well-prepped! You've got the key mental map, so today's lecture will feel much more intuitive.

When you're back, I can help you make a **post-lecture recap or flashcards** to lock it in. Good luck — pay special attention when they show the LoRA update equation or examples of soft prompt embeddings in action.



I just came back from lecture and I have my notes along with some questions.
here are my notes. just read it for now.

Got it — I've read through both notebook pages you uploaded. They're great notes — clear structure from **GPT pretraining → fine-tuning → prompting → PEFT**. I can see the key transitions (e.g. "teacher forcing," "context + last prompt token," and the math around soft prompt parameter counts).

Go ahead and tell me your **questions** — we'll unpack them one at a time using your notes as context.

Why is order and the "data-mix" so important for GPT models?

Excellent question — that's one of the deepest insights in your notes ("*the data-mix matters → order is very important*").

Let's reason it through step by step:

When we say **GPT-style models** are trained to *predict the next token given previous ones*, the model's "worldview" entirely depends on what contexts it sees during training.

Here's the key intuition:

- GPT doesn't learn *structured rules* about language in isolation — it learns *statistical correlations* from **sequences**
- So, if you change the **order or ratio** of data (data mix), you literally change what patterns dominate the training signal.

To make this concrete:

- If you train mostly on **dialogue data**, the model learns to reply conversationally.
- If you train mostly on **code**, it learns structured completion patterns.
- If you shuffle or unbalance data, the model may *forget* or *underweight* some types of reasoning.
- Within each example, **token order** is essential — language meaning depends on sequence. "The dog chased the cat" ≠ "The cat chased the dog."

So the model's next-token prediction is only as good as the *ordered patterns* it's exposed to.

💡 **In short:**

- *Order within examples* → defines linguistic meaning.
- *Order and proportion across datasets (data mix)* → defines *what kind of tasks* the model becomes good at.

Before I move on — want to think about a quick test?
If you had a dataset that's 90% Reddit text and 10% Wikipedia, how would you expect that to shape the model's "personality" or strengths?

ok great thank you. Another thing I didn't really understand is what is the key difference between pretraining and fine tuning?

Excellent follow-up — that's *the* foundational distinction behind all the methods you saw today (ICL, SFT, LoRA, etc.).

Let's reason this out together first —

When you hear **pretraining**, what kind of data and task do you think the model is learning from?
And when you hear **fine-tuning**, what's *different* about that data or objective?

Try describing it in your own words — even roughly.

A good analogy the professor gave in lecture is that pretraining is like slipping a paper with instructions from the other side of a house's door before you actually go in. And then fine-tuning is more about you keeping many parts of the network off and then just learning a small subset of parameters but would like a better explanation.

That's actually a *fantastic* analogy — it captures both the **scope** and **depth** differences between pretraining and fine-tuning. Let's refine it a bit so it fits both conceptually *and* technically.

Pretraining: Building the foundation

- The model learns *general patterns* of language, reasoning, and structure from massive unlabeled text (internet, books, etc.).
- **Objective:** usually *next-token prediction* (a self-supervised task).

- It's like teaching the model to *understand and complete text in general*, not to follow specific instructions.
- In your professor's analogy: the model is *outside the house*, reading countless notes from under the door — it learns how people usually talk, but doesn't yet *live inside* any particular domain or task.

🎯 Fine-Tuning: Customizing behavior

- The model now gets smaller, labeled datasets for a *specific goal* (e.g. summarization, dialogue, classification, instruction following).
- **Objective:** align outputs with the desired style or behavior — often by updating a small part of parameters (*PEFT*).
- This is like stepping *into the house* and rearranging some furniture to suit your needs — not rebuilding the whole structure.

Stage	Data	Goal	Parameters Updated	Example
Pretraining	Billions of unlabeled text	Learn general language model	All	Predict next token
Fine-tuning	Smaller, labeled or curated data	Adapt to specific task or style	All or subset (LoRA, etc.)	Instruction following, chat style

---

💡 So in short:

Pretraining = broad knowledge; Fine-tuning = specialization.

Would you like to go one level deeper and see how *Supervised Fine-Tuning (SFT)* — the term in your notes — fits into this picture?

what essentially is a preprompt and how does it fit in the network?

Excellent — you're picking up on one of the most interesting details from your notes ("pre-prompt length 100… new learnable columns in the embedder").

Let's unpack it carefully.

🧩 What is a *preprompt*?

Think of a **preprompt** (or **soft prompt**) as a *learnable prefix* added to the model's input **before** your real text prompt.

Normally, a GPT input might look like:

```csharp
[Start]  "Translate English to French: cat →"
```

With a **preprompt**, the input becomes:

```csharp
[Soft tokens 1–100]  [Start]  "Translate English to French: cat →"
```

Those 100 "soft tokens" aren't real words — they're **vectors** (embeddings) that the model learns to associate with helpful internal states for the task.

🧠 Where does it fit in the network?

1. The model's first layer — the **embedding layer** — maps each token (word/subword) to a vector.
2. During *soft prompt training*, you add **extra learnable embeddings** to that layer — they're inserted *before* the normal input tokens.
3. Everything else (the transformer layers, attention, etc.) stays frozen.

So the only trainable part is that **block of embeddings** in the input.
It's like adding a short, trainable "instruction code" that the model reads before your text, silently influencing how it processes the rest.

⚙️ Example from your notes

You wrote:

> "Preprompt length = 100... 100 new columns added for the embedder... everything else frozen."

That means:

- The embedding matrix E (shape = vocab × embedding_dim) got **100 new learned vectors** added.
- Those are your soft tokens.
- During training, gradients update only those 100 vectors — not the rest of the network.

💡 In short:

A *preprompt* = learnable prefix embeddings that steer the frozen model toward your task, by conditioning its hidden states right from the input.

Want to visualize it next with a tiny diagram or pseudocode snippet showing exactly *where* those vectors go in the forward pass?

something else i didn't quite understand from lecture is that the gradients flow back by hitting the keys and values back and thus soft prompts work well...

Beautiful — you caught a really subtle and *important* point that your professor slipped in there. Let's unpack this carefully and intuitively, because it explains **why soft prompts even work**, despite being so tiny.

🧠 Step 1. Recall how attention works

Each transformer layer uses **queries (Q)**, **keys (K)**, and **values (V)** derived from the token embeddings.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

So:

- Each token's **query** looks at other tokens' **keys** and **values** to decide *what to attend to*.
- The *influence* between tokens happens through those K/V projections.

🔁 Step 2. Where soft prompts sit

The **soft prompt vectors** are inserted **at the beginning** of the input sequence.
When the model processes attention, all later tokens (like your actual text) can *attend to* those prefix embeddings.

So, those soft prompt vectors produce their own **K** and **V** representations —
→ the later real tokens' **queries** can use them,
→ meaning the prompt can *inject information or bias* into every subsequent attention step.

⚙️ Step 3. Why gradient flow through K/V is crucial

When training the soft prompt:

- The model's weights are frozen — you can't change Q, K, or V linear layers.
- But you *can* change the *inputs* that feed into them — i.e., your soft prompt embeddings.

So, during backpropagation:

1. The loss gradient flows backward through the attention mechanism.
2. It reaches the **keys and values** produced by the soft prompt vectors.
3. That updates the soft prompt embeddings to become *better at influencing* how later tokens attend.

That's why your professor said:

> "Gradients hit the keys and values back — that's how soft prompts work."

They're not learning in the *output* sense (like adjusting logits); they're learning to *shape attention* through their position as K/V sources.

💡 **In essence:**
Soft prompts work because they become *trainable keys and values* that the model learns to attend to — they condition the computation path of the frozen transformer.

Would you like me to show this with a small sketch or pseudocode trace of how the gradient flows during one attention pass? It makes that "gradient hitting K/V" idea very visual.

yes. i'd like a small sketch please

Perfect — let's make this *visual but simple*.

🧩 Imagine your model input:

```
[Soft1] [Soft2] [Soft3] [Text1] [Text2] [Text3]
```

The first few tokens (Soft1–3) are your **soft prompt embeddings**, and the rest are your real text.

🧠 Inside one transformer layer:

Each token gets turned into **Q, K, V** (queries, keys, values):

```
Soft1 → K,V
Soft2 → K,V
Soft3 → K,V
Text1 → Q,K,V
Text2 → Q,K,V
Text3 → Q,K,V
```

When the model runs attention, *each text token's query* looks at **all** keys/values — including the soft prompt ones.

So visually:

css Copy code

```
          ┌──────────────────────────────────────┐
          │  Attention between tokens            │
          │                                      │
Text3 Q → │  attends to [Soft1 K,V], [Soft2 K,V], [Soft3 K,V], ... │
          └──────────────────────────────────────┘
```

That means the soft prompts can influence how every later token's representation is computed.

🔁 Now during training (backprop):

Let's say we have a loss L on the model's final output.

Gradients flow backward like this:

mathematica Copy code

```
Loss L
  ↓
Output logits
  ↓
Transformer layers
  ↓
Attention (Q, K, V)
  ↓
Soft prompt K, V embeddings  ←——  GRADIENT HITS HERE
```

Since the main network is frozen, gradients stop at the **soft prompt embeddings** — and those embeddings get updated.

Over time, they learn values for K and V that steer attention in helpful directions.

🧩 In short:

Soft prompts **work** because they become *learnable "memory slots"* at the start of the sequence —
attention flows **through** them forward, and gradients flow **back to** them in reverse.

Would you like me to draw this as a small *diagram image* (arrows and labeled boxes, showing forward + backward flows)? It'd look like a mini map of attention with gradients.

ok to wrap it up give me a 5 question conceptual multiple choice post-lecture quiz.

Perfect wrap-up move — post-lecture quizzes are *the* best way to make knowledge stick.
Here's your **5-question conceptual quiz** on *In-context learning, prompting, and PEFT (Soft Prompting + LoRA)*.

🧠 Post-Lecture Concept Quiz

1. In-context learning primarily differs from fine-tuning because:
A) It updates a subset of the model's parameters.
B) It uses task-specific labeled data.
C) It does not update any parameters — learning happens through examples in the input.
D) It modifies the embedding matrix only.

---

2. What is the main goal of Parameter-Efficient Fine-Tuning (PEFT)?

A) To compress model weights for faster inference.

B) To train large models using fewer updated parameters while preserving performance.

C) To retrain all layers but with a smaller learning rate.

D) To replace pretraining entirely.

---

3. In soft prompting, where do the new learnable parameters live?

A) Inside the feedforward layers.

B) In the attention heads.

C) As additional embeddings prepended to the input sequence.

D) In the output projection layer.

---

4. Why do gradients "hitting the keys and values" make soft prompts effective?

A) Because it allows the model to learn which soft tokens to ignore.

B) Because it updates the soft prompt embeddings to control attention flows through K/V channels.

C) Because it retrains the attention weight matrices.

D) Because it freezes the embeddings and unfreezes attention layers.

---

5. Compared to soft prompting, LoRA generally:

A) Trains faster but performs worse on complex tasks.

B) Requires full fine-tuning of all model weights.

C) Inserts low-rank adapters inside transformer layers for richer adaptation.

D) Modifies only the output layer.

Would you like me to include a short **answer key with reasoning** for you to check after you try it yourself?

1. C
2. B

3. C
4. B
5. C

🔥 Excellent — that's a perfect sweep.
Let's confirm each one quickly with reasoning:

---

1. ✅ C — In-context learning doesn't update weights.
It "learns" by showing examples in the prompt; all parameter weights remain frozen.

2. ✅ B — PEFT minimizes trainable parameters.
It's about adapting massive pretrained models efficiently, often updating <1% of weights.

3. ✅ C — Soft prompts = new embeddings prepended.
They act like learnable "virtual tokens" that guide the model before the actual task input.

4. ✅ B — Gradients hitting K/V paths tune attention.
Soft prompts influence which parts of the model's internal representations the query attends to.

5. ✅ C — LoRA adds low-rank adapters in transformer layers.
Instead of new tokens, LoRA changes how intermediate layers adapt, giving richer capacity than soft prompts.